



## Lab 05: Local Search Algorithms

**Objective:** Investigate how local search algorithms navigate a fitness landscape without retaining a search path. You will use the **Optimization Algorithm Simulator** from the lecture to observe how Hill Climbing, Simulated Annealing and Local Beam Search behave under different parameter settings, explain the trade-offs you observe, and then implement the generate–evaluate–prune cycle of Local Beam Search in Python. Total: **5.0 marks**.

|                     |                |                   |            |
|---------------------|----------------|-------------------|------------|
| <b>Student Name</b> | Peiris M D D P | <b>Student ID</b> | IT24100532 |
|---------------------|----------------|-------------------|------------|

### Introduction

Local search algorithms do not build or store a path to the goal. They hold one current state (or a small set of states), look only at that state's immediate neighbours, and move according to a selection rule. Because the path is discarded, memory use is constant, which makes these methods suitable for very large or continuous state spaces where systematic search is infeasible. The cost of that efficiency is completeness: the algorithm can settle on a *local* optimum and stop.

**Hill Climbing (Greedy):** always move to the best-scoring neighbour; halt when no neighbour improves on the current state.

**Simulated Annealing:** accept an improving move always, and accept a worsening move with probability  $P = e^{-(\Delta E / T)}$ , where the temperature  $T$  falls over time according to a cooling schedule.

**Local Beam Search:** keep  $k$  states, generate all their successors into one shared pool, and retain the best  $k$  of that pool for the next iteration.

### Lab Activity

This is an individual activity. Complete each section on your own and type your answers directly into the boxes below.

Use the Optimization Algorithm Simulator provided in the lecture for Part A. Answer the MCQs in Part B directly in this sheet. For Part C, copy the skeleton code into your local IDE (PyCharm, VS Code, Jupyter or Colab), complete and test it, then paste the working code back into the box provided. Use **Print / Save as PDF** at the top of the page to export your completed sheet for submission.

## Section 1: Part A — Simulation Analysis & Parameter Tweaks (2.0 Marks)

### 1. The Hill Climbing trap and step size (0.5 marks)

Set the simulator to **Hill Climbing (Greedy)** with **Starting X = 100**.

**Tweak 1:** run with **Step Size = 2** and note where the search becomes trapped.

**Tweak 2:** change **Step Size = 20** and run again. Does it step over the local maximum?

| Run            | Final X | Peak reached (local / global) |
|----------------|---------|-------------------------------|
| Step Size = 2  | 144     | Local                         |
| Step Size = 20 | 140     | Local                         |

**Explain the trade-off:** how does a larger step size help the search escape a local optimum, and what is the dangerous drawback of setting the step size too large as the search nears the global maximum?

A larger step size lets the search look further ahead, so it can jump over the dip between hills and escape a local maximum. The drawback is overshooting: near the peak, big steps skip past the top instead of landing on it. In our runs, step 2 settled precisely at  $x = 144$  (266.06) but was trapped on a local peak. Step 20 still didn't escape, and it ended lower at  $x = 140$  (265.29) because it could only land at 140 or 160, never at the true top.

### 2. Initial condition sensitivity (0.5 marks)

Stay on **Hill Climbing** and set **Step Size = 5**.

**Tweak 1:** start at **X = 50**, run, and record the final peak.

**Tweak 2:** start at **X = 850**, run, and record the final peak.

| Starting X | Final X | Final objective value |
|------------|---------|-----------------------|
| 50         | 50      | 248.38                |
| 850        | 765     | 281.38                |

**Observe & explain:** compare the two final states. Why is greedy local search so sensitive to its initial configuration, and how does Random-Restart Hill Climbing mitigate this specific vulnerability?

The two runs ended on completely different peaks. Starting at  $x = 50$ , the search stopped immediately with 248.38, because it was already on top of a small local peak and no neighbour was better. Starting at  $x = 850$ , it climbed left to  $x = 765$  and reached 281.38. Greedy hill climbing only moves uphill, so it just climbs whichever hill it starts on. The starting point alone decides the result. Random-Restart Hill Climbing reduces this by running the search many times from random starting points and keeping the best result, which increases the chance that one run starts on the global maximum's hill.

### 3. Simulated Annealing: the cooling schedule (0.5 marks)

Switch to **Simulated Annealing** with **Starting X = 100** and **Initial Temp = 100**.

**Tweak 1: Cooling Rate = 0.50** (fast cooling). Run the simulation.

**Tweak 2: Cooling Rate = 0.99** (slow cooling). Run the simulation.

| Cooling rate | Final X / value | Behaviour observed                                                                                    |
|--------------|-----------------|-------------------------------------------------------------------------------------------------------|
| 0.50 (fast)  | 144 / 265       | fast and settled very quickly. small number of steps                                                  |
| 0.99 (slow)  | 144 / 265.37    | very slow and high step count. at first started accepting most worse moves but later it rejected them |

**Explain the impact:** how does the cooling rate control the balance between exploration (random walk) and exploitation (greedy climbing)? Referring to  $P = e^{-(\Delta E / T)}$ , state mathematically what happens to the acceptance probability for a worsening move when the temperature falls too rapidly.

The cooling rate decides how long the search keeps exploring before it turns greedy. With slow cooling (0.99),  $T$  stays high for a long time, so the search accepts many worse moves and wanders like a random walk (exploration) before gradually turning into greedy climbing (exploitation). With fast cooling (0.50),  $T$  collapses within a few steps, so the search becomes greedy almost immediately and stays stuck near where it started.

#### 4. Local Beam Search: the loss of diversity (0.5 marks)

Set the simulator to **Local Beam Search** with **Beam Width ( $k$ ) = 3** and run from a random start. You will most likely see all three agents converge onto the same peak within a few iterations, leaving the rest of the landscape unexplored.

**Observe & explain:** using the algorithm's successor-selection logic, explain mathematically why this loss of diversity occurs. In your answer, state how the successors of all  $k$  states are pooled and how the top  $k$  are chosen.

In each round, every agent generates its neighbours, and they all go into one shared pool. Only the best 3 in the whole pool survive, and no agent is guaranteed a place. If one agent is on a higher spot, its neighbours outscore everyone else's and take all 3 places, so the other agents are dropped. In our run, all three places held 248 by step 2, meaning they all came from one agent. After that, the three climbed together and got stuck on the same local peak (266). The higher peak at  $x = 765$  (281.38) was never explored.

## Section 2: Part B — Theoretical Foundations (MCQs) (1.5 Marks)

Select one option per question and type the letter in the answer box. 0.5 marks each.

### 5. Which statement best distinguishes Local Search from standard pathfinding (such as A\*)?

- A) Local search uses heuristics, while standard pathfinding does not.
- B) Local search aims to find the optimal sequence of actions, while pathfinding only cares about the final state.
- C) Standard pathfinding retains paths in memory to backtrack, whereas local search operates

using only the current node (and its neighbours), focusing on the state rather than the path.  
D) Local search algorithms are guaranteed to be complete and optimal in continuous spaces.

C

**6. In Simulated Annealing the probability of accepting a worse state is  $P = e^{-(\Delta E / T)}$ . As the temperature  $T$  approaches 0, what happens to the behaviour of the algorithm?**

- A) It behaves exactly like a purely random walk.
- B) It behaves exactly like greedy Hill Climbing, accepting only better states.
- C) It accepts all states regardless of their fitness evaluation.
- D) The algorithm terminates automatically because  $\Delta E$  becomes undefined.

B

**7. How does Stochastic Beam Search solve the loss-of-diversity problem found in standard Local Beam Search?**

- A) By choosing the next  $k$  states randomly, with probability proportional to their fitness, rather than strictly choosing the absolute best  $k$ .
- B) By increasing the beam width  $k$  dynamically whenever a local minimum is detected.
- C) By forcing agents to move in opposite directions (repulsion) when they get too close to each other.
- D) By completely resetting the temperature whenever all  $k$  states converge.

A

### Section 3: Part C — Guided Implementation: Local Beam Search (1.5 Marks)

#### Development instructions:

1. Copy the skeleton below into your local Python IDE (Jupyter, VS Code or Colab).
2. Complete the logic for **generating**, **evaluating** and **pruning** the neighbours.
3. Run the script. The `__main__` block contains a test case (a mathematical parabola); a correct implementation prints a success message.
4. Once it passes, paste your completed code into the box below.

#### Skeleton code

```

import random

def local_beam_search(initial_states, k, max_iters, objective_fn):
    """
    initial_states: list of k randomly generated starting positions
    k: the beam width (number of states to keep)
    objective_fn: the function we are trying to maximize
    """
    current_states = initial_states

    for iteration in range(max_iters):
        all_successors = [] # Will hold tuples of (state, fitness)

        # --- STEP 1, 2 & 3: Generate, Evaluate, and Pool Neighbors ---
        for state in current_states:
            # Generate 5 random neighbors for this specific state
            for _ in range(5):
                # HINT: neighbor = state + random.uniform(-10, 10)
                # [WRITE CODE HERE]

                # HINT: calc fitness -> fit = objective_fn(neighbor)
                # [WRITE CODE HERE]

                # HINT: append to pool -> all_successors.append((neighbor, fit))
                # [WRITE CODE HERE]
            pass

        # --- STEP 4: Prune the Beam (The Cut) ---
        # HINT 1: Sort the pool descending by fitness.
        # all_successors.sort(key=lambda x: x[1], reverse=True)
        # [WRITE CODE HERE]

        # HINT 2: Slice the top 'k' items and extract just the state (
        # current_states = [item[0] for item in all_successors[:k]]
        # [WRITE CODE HERE]

    return max(current_states, key=objective_fn)

# =====
# LOCAL TEST RUNNER - DO NOT MODIFY BELOW THIS LINE
# =====
if __name__ == "__main__":
    # Fix the random seed for reproducible testing
    random.seed(42)

```

**Paste your completed, tested code here**

```

import random

def local_beam_search(initial_states, k, max_iters, objective_fn):
    """
    initial_states: list of k randomly generated starting positions
    k: the beam width (number of states to keep)
    objective_fn: the function we are trying to maximize
    """
    current_states = initial_states

    for iteration in range(max_iters):
        all_successors = []    # Will hold tuples of (state, fitness)

        # --- STEP 1, 2 & 3: Generate, Evaluate, and Pool Neighbors ---
        for state in current_states:
            # Generate 5 random neighbors for this specific state
            for _ in range(5):
                neighbor = state + random.uniform(-10, 10)
                fit = objective_fn(neighbor)
                all_successors.append((neighbor, fit))

            # --- STEP 4: Prune the Beam (The Cut) ---
            all_successors.sort(key=lambda x: x[1], reverse=True)
            current_states = [item[0] for item in all_successors[:k]]

    return max(current_states, key=objective_fn)

# =====
# LOCAL TEST RUNNER - DO NOT MODIFY BELOW THIS LINE
# =====

if __name__ == "__main__":
    # Fix the random seed for reproducible testing
    random.seed(42)

    # NOTE: stand-in test (the lab sheet's runner is cut off after
    random.seed(42)).
    # Parabola with a single global maximum of 0 at x = 50.
    objective = lambda x: -(x - 50) ** 2
    k = 3
    starts = [random.uniform(-500, 500) for _ in range(k)]
    best = local_beam_search(starts, k=k, max_iters=100,
    objective_fn=objective)

```

```
print(f"Initial states: {[round(s, 2) for s in starts]}")
print(f"Best state found: x = {best:.4f}, f(x) =
{objective(best):.4f}")
if abs(best - 50) < 5:
    print("SUCCESS: Local Beam Search converged to the global
maximum (x ~ 50).")
else:
    print("FAILED: Beam did not converge to the global maximum.")
```

## Test output

Paste the exact output printed by the test runner, including the success message and the best state returned.

```
Initial states: [139.43, -474.99, -224.97]
Best state found: x = 52.5552, f(x) = -6.5293
SUCCESS: Local Beam Search converged to the global maximum (x ~ 50).
```

## Discussion

Local Beam Search with  $k = 3$  is not the same as running Hill Climbing three times independently. Explain the difference and say which one preserves more diversity, and why.

Beam search vs. three separate hill climbs: Three separate hill climbs never share anything. Each one climbs its own hill, so they can end on three different peaks. Local beam search puts all the agents' neighbours into one shared pool and keeps only the best 3. If one agent is doing better, its neighbours take over all the places, and the whole beam ends up on one hill. In our run, all three were on the same 266 peak. So running hill climbing three times independently keeps more diversity, because no run can push the others out.

Your objective function is expensive to evaluate and you are given a fixed budget of evaluations. Which algorithm from this lab would you choose, and how would you set its parameters? Justify your choice using your Part A observations.

Fixed evaluation budget: I'd choose Hill Climbing with random restarts, using a medium step size (about 5–10) and as many restarts as the budget allows. Part A showed that hill climbing is very cheap: it finished in 10–23 steps, and the start at 850 found the best result of the whole lab (281.38). Its weakness is depending on the starting point, and restarts fix exactly that. Simulated annealing with slow cooling used 528 steps and still ended at only 265.37. Local beam search spent effort on

three agents that all collapsed onto the same local peak. For the same budget, several quick climbs from different starting points give the best chance of reaching the global peak.



**FACULTY OF COMPUTING**